UNIVERSIDADE FEDERAL DE LAVRAS
Engenharia de Software

Rafaella Tavares Moreira

TRABALHO DE PROSPECÇÃO TECNOLÓGICA

Outubro/2026

1. INTRODUÇÃO E CONTEXTUALIZAÇÃO

O avanço da inteligência artificial (IA), especialmente dos Large Language Models (LLMs), tem ampliado as possibilidades de desenvolvimento de aplicações capazes de compreender e gerar textos, responder a perguntas e auxiliar na execução de diferentes tarefas. Esses modelos vêm sendo incorporados a sistemas de atendimento, assistentes virtuais, ferramentas de análise de informações e outras soluções computacionais. Entretanto, a utilização de LLMs em aplicações reais envolve desafios que vão além da escolha do modelo e da implementação de suas funcionalidades, exigindo mecanismos que garantam qualidade, segurança, desempenho e confiabilidade durante sua operação.

Diferentemente dos sistemas de software tradicionais, cujos comportamentos podem ser definidos e testados a partir de regras explícitas, as aplicações baseadas em LLMs apresentam características probabilísticas, podendo produzir respostas diferentes para uma mesma entrada. Além disso, esses modelos estão sujeitos a problemas como a geração de informações incorretas, conhecida como alucinação, a exposição de dados sensíveis e a ocorrência de respostas inadequadas. Outros fatores, como o custo das chamadas às interfaces de programação de aplicações (Application Programming Interfaces – APIs), o tempo de resposta e a necessidade de atualização dos modelos, também dificultam a manutenção dessas soluções. Dessa forma, transformar um protótipo funcional em uma aplicação estável, escalável e segura exige práticas específicas de desenvolvimento, avaliação e acompanhamento contínuo.

Nesse contexto, surge o LLMOps (Large Language Model Operations), abordagem voltada ao gerenciamento, à implantação, ao monitoramento e à otimização de aplicações que utilizam grandes modelos de linguagem. Seu objetivo é estabelecer práticas e processos que permitam integrar esses modelos aos ambientes de produção de maneira controlada, abrangendo não apenas o modelo em si, mas também os componentes envolvidos na interação com o usuário. Entre suas principais atividades estão o gerenciamento e o versionamento de prompts, a avaliação da qualidade das respostas, o monitoramento de desempenho e custos, a implementação de mecanismos de segurança e a definição de estratégias para lidar com falhas. Quando necessário, também pode envolver técnicas como a geração aumentada por recuperação (Retrieval-Augmented Generation – RAG), que permite fornecer informações relevantes de fontes externas ao modelo durante a geração das respostas.

A aplicação dessas práticas é relevante para a Engenharia de Software, pois contribui para tornar o desenvolvimento e a manutenção de sistemas baseados em IA generativa mais sistemáticos e rastreáveis. O LLMOps permite acompanhar alterações nos modelos e nos

prompts, identificar possíveis degradações na qualidade das respostas e avaliar os efeitos de mudanças antes de disponibilizá-las aos usuários. Além disso, favorece o controle do consumo de recursos computacionais, a redução de custos operacionais e a adoção de medidas para prevenir problemas relacionados à privacidade e à segurança. Dessa maneira, aproxima o desenvolvimento de aplicações com LLMs das práticas de operação e manutenção já consolidadas na engenharia de software, considerando as particularidades desses modelos.

Diante desse cenário, o LLMOps apresenta-se como uma abordagem tecnológica relevante para a construção e a sustentação de aplicações baseadas em grandes modelos de linguagem. Sua investigação permite compreender como diferentes ferramentas, técnicas e processos podem contribuir para superar limitações relacionadas à confiabilidade, à escalabilidade, à observabilidade e à governança dessas aplicações. Assim, o estudo desse tema busca explorar os recursos e as práticas associados ao LLMOps, bem como suas possibilidades de aplicação no desenvolvimento de software, considerando os desafios envolvidos na utilização de inteligência artificial generativa em ambientes reais.

2. FUNDAMENTAÇÃO TEÓRICA

2.1 HISTÓRICO E EVOLUÇÃO DO LLMOps

O surgimento do LLMOps está relacionado à evolução das práticas de desenvolvimento e operação de software. No final da década de 2000, o DevOps ganhou espaço ao aproximar as equipes de desenvolvimento e operações, promovendo a automação de testes, a integração contínua e a implantação de sistemas. Posteriormente, com a expansão do aprendizado de máquina, surgiu o MLOps, que adaptou esses princípios para gerenciar o ciclo de vida dos modelos, incluindo treinamento, versionamento e monitoramento (DEY, 2025).

Com a popularização dos Grandes Modelos de Linguagem (Large Language Models – LLMs), especialmente a partir de 2022, surgiram novos desafios relacionados à implantação e à manutenção de aplicações de inteligência artificial generativa. Nesse contexto, o LLMOps consolidou-se como um conjunto de práticas voltadas à operação desses sistemas, considerando aspectos como gerenciamento de prompts, avaliação das respostas, monitoramento de custos e desempenho e mecanismos de segurança. Dessa forma, o LLMOps amplia as práticas anteriores para atender às particularidades dos modelos de linguagem em ambientes de produção (RED HAT, 2024).

2.2 ARQUITETURA E FUNCIONAMENTO

O LLMOps não possui uma arquitetura única, pois seus componentes variam conforme as necessidades da aplicação. De maneira geral, seu funcionamento envolve etapas de processamento das solicitações, geração de respostas e monitoramento contínuo, permitindo que os modelos de linguagem sejam utilizados de forma controlada em ambientes de produção (MICROSOFT, s.d.; RED HAT, 2024).

O processo começa com o recebimento da solicitação do usuário, denominada prompt. Na etapa de pré-processamento, a entrada é validada e preparada para o processamento, podendo incluir verificações de formato, tamanho e segurança. Em seguida, o sistema pode complementar a solicitação com informações do histórico da conversa ou de fontes externas, como documentos e bancos de dados. Essa etapa, conhecida como contextualização, permite fornecer informações relevantes ao modelo. Uma das técnicas utilizadas para esse propósito é o RAG (Retrieval-Augmented Generation), que recupera informações de uma base de conhecimento e as incorpora ao contexto enviado ao modelo (DEY, 2025; RED HAT, 2024).

Após a preparação da entrada, o prompt é encaminhado ao modelo de linguagem, que realiza a inferência e gera uma resposta com base nas instruções e nas informações recebidas. Antes de disponibilizar o resultado ao usuário, a aplicação pode executar verificações de segurança e consistência, além de realizar ajustes no formato da resposta. Essas medidas ajudam a reduzir riscos, como a apresentação de conteúdos inadequados e de informações não fundamentadas, embora não eliminem completamente a possibilidade de erros.

Paralelamente ao processamento das solicitações, o LLMOps realiza o monitoramento da aplicação. Nessa etapa, são acompanhados indicadores como tempo de resposta, consumo de tokens, custos operacionais e qualidade das respostas. Os registros gerados permitem identificar falhas, investigar problemas e avaliar se o sistema atende aos requisitos estabelecidos. Com base nessas informações, podem ser realizadas melhorias, como ajustes nos prompts, alterações nas estratégias de recuperação de informações ou substituição do modelo utilizado.

Dessa forma, o LLMOps estabelece um ciclo contínuo de operação, avaliação e aprimoramento, abrangendo desde o recebimento da solicitação até a análise dos resultados produzidos. A Figura 1 apresenta uma representação simplificada desse funcionamento.

Figura 1 – Fluxo geral de uma aplicação baseada em LLM com práticas de LLMOps



Fonte: *Adaptado de Microsoft (s.d.), com tradução e modificações realizadas com auxílio do Google Gemini (2026).*

O fluxo apresentado é ilustrativo, uma vez que as etapas podem variar conforme a aplicação. O monitoramento e a avaliação acompanham o funcionamento do sistema e fornecem subsídios para aperfeiçoar seus componentes, contribuindo para a qualidade, a eficiência e a segurança da solução.

3. ANÁLISE COMPARATIVA

3.1 COMPARAÇÃO COM OUTRAS TECNOLOGIAS

O LLMOps representa uma evolução das práticas de operação de software e inteligência artificial, adaptada às particularidades dos grandes modelos de linguagem. Para compreender sua relevância, é importante compará-lo com o DevOps e o MLOps, abordagens que compartilham princípios de automação, monitoramento e melhoria contínua, mas possuem objetivos operacionais distintos (DEY, 2025; RED HAT, 2024).

O DevOps concentra-se na integração entre desenvolvimento e operações de software, utilizando práticas como integração contínua, implantação contínua e monitoramento de serviços. Embora essas práticas também sejam utilizadas em aplicações com inteligência artificial, elas não contemplam, isoladamente, todas as necessidades dos LLMs, como avaliação semântica das respostas, gerenciamento de prompts e identificação de alucinações.

O MLOps, por sua vez, amplia os princípios do DevOps para o ciclo de vida de modelos de aprendizado de máquina, incluindo preparação de dados, treinamento, avaliação, versionamento e monitoramento. Entretanto, em aplicações baseadas em LLMs, é necessário

considerar aspectos adicionais, como o controle do consumo de tokens, a variabilidade das respostas, a integração com bases de conhecimento e os riscos de manipulação das instruções do modelo (DEY, 2025; RED HAT, 2024).

Nesse contexto, o LLMOps incorpora práticas das duas abordagens e acrescenta mecanismos específicos para gerenciar aplicações de linguagem generativa. A Tabela 1 apresenta as principais diferenças entre essas tecnologias.

Tabela 1 – Comparação entre DevOps, MLOps e LLMOps

Critério	**DevOps**	**MLOps**	**LLMOps**
Foco principal	Desenvolvimento e operação de software	Ciclo de vida de modelos de ML	Operação de aplicações com LLMs
Artefatos gerenciados	Código e pacotes de software	Dados, modelos e experimentos	Modelos, prompts, contexto e componentes de recuperação
Avaliação	Testes de software e desempenho	Métricas como acurácia e precisão	Qualidade, relevância e segurança das respostas
Monitoramento	Disponibilidade e falhas	Desempenho do modelo e alterações na distribuição dos dados	Qualidade das respostas, latência e consumo de tokens
Principais desafios	Integração e implantação	Reprodutibilidade e degradação dos modelos	Alucinações, custos, segurança e variabilidade das saídas

Fonte: *Elaborado pela autora com base nas referências (DEY, 2025; RED HAT, 2024).*

A comparação demonstra que essas abordagens são complementares, e não necessariamente concorrentes. O DevOps continua sendo importante para a infraestrutura e a entrega do software, enquanto o MLOps oferece práticas para gerenciar modelos de aprendizado de máquina. O LLMOps, por sua vez, acrescenta recursos necessários para acompanhar o comportamento e a qualidade das aplicações baseadas em grandes modelos de linguagem.

3.2 VANTAGENS E LIMITAÇÕES

A principal vantagem do LLMOps é permitir que aplicações baseadas em grandes modelos de linguagem sejam gerenciadas de maneira mais sistemática. O versionamento de prompts e modelos, associado à avaliação contínua, facilita a identificação de alterações que prejudicam a qualidade das respostas e permite recuperar configurações anteriores quando necessário. Além disso, o monitoramento do consumo de tokens e da latência auxilia na otimização dos custos e do desempenho (DEY, 2025; RED HAT, 2024).

Outra vantagem é a possibilidade de integrar mecanismos de segurança, monitoramento e recuperação de falhas ao longo da operação. Em aplicações corporativas, essas práticas contribuem para reduzir riscos relacionados à exposição de informações sensíveis e à geração de conteúdo inadequado. A integração com técnicas como RAG também permite utilizar fontes externas de informação para contextualizar as respostas, favorecendo aplicações que dependem de documentos ou conhecimentos específicos (RED HAT, 2024).

Apesar desses benefícios, a adoção do LLMOps apresenta limitações. A avaliação da qualidade das respostas pode ser complexa, pois nem sempre existe uma única resposta correta ou uma métrica capaz de representar adequadamente sua utilidade. Além disso, a implementação de monitoramento, mecanismos de segurança e infraestrutura de avaliação exige recursos técnicos e financeiros, podendo representar um custo elevado para projetos de pequeno porte (DEY, 2025).

Outra limitação é que o LLMOps não elimina os problemas inerentes aos modelos de linguagem. Mesmo com mecanismos de controle, ainda podem ocorrer alucinações, respostas inconsistentes e falhas diante de entradas maliciosas. Da mesma forma, o uso de modelos externos pode gerar dependência de provedores, restrições de licenciamento e preocupações com a privacidade dos dados (DEY, 2025; RED HAT, 2024).

Portanto, a adoção do LLMOps deve considerar as necessidades e os riscos de cada aplicação. Em sistemas corporativos, assistentes virtuais e soluções que processam informações sensíveis ou atendem muitos usuários, suas práticas podem ser fundamentais para garantir maior controle e rastreabilidade. Já em protótipos simples, de baixo uso e sem requisitos rigorosos de disponibilidade, a implementação de uma estrutura completa de LLMOps pode ser desproporcional, sendo mais adequado começar com mecanismos básicos de avaliação, controle de custos e segurança e ampliá-los conforme a solução evolui.

4. APLICAÇÃO PRÁTICA

4.1 AMBIENTE DE DESENVOLVIMENTO

Para demonstrar os conceitos de LLMOps, foi desenvolvido um pipeline simplificado de avaliação e monitoramento de modelos de linguagem utilizando Python. O projeto combina um notebook Jupyter, para a execução e análise dos experimentos, com uma interface desenvolvida em Streamlit, que permite interagir com o modelo e visualizar os resultados.

As principais ferramentas e bibliotecas utilizadas foram:

- Python 3.10 ou superior: linguagem utilizada no desenvolvimento da aplicação e na implementação da lógica de avaliação.
- Jupyter Notebook: ambiente para executar os experimentos, avaliar as respostas e comparar os resultados.
- Streamlit: biblioteca utilizada para criar a interface de interação com o sistema.
- API de um modelo de linguagem (LLM): responsável por receber as perguntas e gerar as respostas. A configuração é feita por meio de variáveis de ambiente, permitindo definir o modelo e o provedor utilizado.
- Pandas: utilizado para organizar, processar e analisar os dados das execuções.
- Matplotlib: utilizado para gerar gráficos comparativos das métricas.
- python-dotenv: utilizado para carregar as configurações armazenadas no arquivo .env.
- CSV: formato utilizado para armazenar o conjunto de perguntas de teste e o histórico das execuções.

O projeto foi organizado em arquivos com responsabilidades distintas. O *llmops_core.py* concentra a lógica principal, incluindo o gerenciamento dos prompts, a chamada ao modelo, o cálculo das métricas e o registro das execuções. O arquivo *app.py* contém a interface Streamlit, enquanto o notebook *llmops_demo.ipynb* reúne os experimentos realizados. O arquivo perguntas.csv armazena oito perguntas de computação, suas respostas esperadas e palavras-chave para avaliação.

A configuração da API é realizada por meio de um arquivo .env, no qual são definidos a chave de acesso, o modelo e os preços utilizados no cálculo do custo estimado. Essa separação permite alterar as configurações sem modificar diretamente o código-fonte. Este arquivo *.env* é disponibilizado no repositório como *.env.example,* a chave de API é uma credencial privada e não deve ser incluída no repositório público, desta forma o arquivo é fornecido apenas como exemplo, contendo apenas nomes de variáveis e valores fictícios para que possa ser modificado por quem for executar o projeto, incluindo sua chave própria. O

projeto também possui um modo simulado, que possibilita testar o fluxo da aplicação sem realizar chamadas à API.

As dependências necessárias para executar o projeto estão descritas no arquivo requirements.txt, que auxilia na configuração e na reprodução do ambiente de desenvolvimento.

4.2 DEMONSTRAÇÃO DO CÓDIGO

A aplicação foi desenvolvida para demonstrar um fluxo reduzido de LLMOps, desde a geração de uma resposta até o registro, a avaliação e a comparação dos resultados. Para isso, foram implementadas duas versões de prompt, mantendo o mesmo modelo de linguagem durante os experimentos. Essa escolha permite observar os efeitos da alteração das instruções fornecidas ao modelo.

4.2.1 VERSIONAMENTO DE PROMPTS

No arquivo *llmops_core.py,* os prompts são organizados em um dicionário chamado *PROMPTS*, que identifica cada versão e armazena suas instruções:

```
18  PROMPTS = {
19      "A": {
20          "versao": "A-v1",
21          "texto": "Responda à pergunta do usuário.",
22      },
23      "B": {
24          "versao": "B-v1",
25          "texto": (
26              "Responda com clareza e precisão, explique os conceitos importantes "
27              "e utilize um exemplo simples quando ajudar na compreensão. "
28              "Se não souber a resposta, informe isso."
29          ),
30      },
31  }
```

A versão A utiliza uma instrução simples, enquanto a versão B estabelece critérios adicionais para a elaboração da resposta. O versionamento permite identificar qual conjunto de instruções foi utilizado em cada execução e comparar seus resultados posteriormente. Dessa forma, uma alteração no prompt passa a ser tratada como uma mudança que pode ser avaliada, em vez de uma modificação sem histórico.

4.2.2 CHAMADA DO MODELO E MONITORAMENTO DAS MÉTRICAS

A função *chamar_llm()*, também presente no arquivo *llmops_core.py,* é responsável por enviar a pergunta e o prompt selecionado ao modelo. Durante essa operação, o sistema mede o tempo de resposta e coleta informações sobre o consumo de tokens.

O trecho a seguir ilustra a medição da latência:

```
153         inicio = time.perf_counter()
154         try:
155             resp = cliente.chat.completions.create(
156                 model=cfg["modelo"],
157                 messages=mensagens,
158                 temperature=temperatura,
159             )
160             latencia = time.perf_counter() - inicio
161             texto = resp.choices[0].message.content or ""
```

A função registra o instante anterior à requisição e calcula o tempo decorrido após o retorno da API. Em seguida, obtém o conteúdo da resposta. Quando a API disponibiliza os dados de utilização, o sistema registra os tokens de entrada e saída. Esses dados permitem acompanhar aspectos importantes da execução, como latência e consumo de recursos. O custo é calculado separadamente, com base na quantidade de tokens e nos preços configurados para o modelo. Assim, o projeto demonstra como a observabilidade pode auxiliar na análise do desempenho e dos custos de uma aplicação com LLM.

4.2.3 REGISTRO E RASTREABILIDADE DAS EXECUÇÕES

Após a chamada ao modelo, a função *executar_e_registrar()* reúne os dados produzidos e registra uma nova linha no arquivo *execucoes.csv*. Entre as informações armazenadas estão a identificação da execução, a data e hora, a pergunta, a versão do prompt, o modelo utilizado, a resposta, a latência, os tokens, o custo estimado e as avaliações de qualidade.

O registro é realizado por meio da função *registrar_execucao():*

```
186 def registrar_execucao(linha):
187     """Acrescenta uma execução ao CSV """
188     df = pd.DataFrame([linha]).reindex(columns=COLUNAS)
189     novo = not LOG_PATH.exists()
190     df.to_csv(LOG_PATH, mode="a", header=novo, index=False, encoding="utf-8")
```

O código transforma os dados da execução em um DataFrame e os acrescenta ao arquivo *CSV*. O cabeçalho é escrito somente quando o arquivo ainda não existe, permitindo manter um histórico acumulado. Esse mecanismo contribui para a rastreabilidade, pois possibilita consultar as execuções anteriores e identificar quais configurações foram utilizadas em cada resposta.

4.2.4 AVALIAÇÃO E COMPARAÇÃO DAS VERSÕES

A avaliação da qualidade é realizada de duas formas complementares. A primeira é automática, por meio da função *cobertura_palavras_chave()*, que verifica a presença de palavras-chave esperadas na resposta e converte a proporção encontrada para uma escala de

zero a cinco. A segunda é manual, permitindo atribuir uma nota de zero a cinco após comparar a resposta gerada com a resposta esperada.

No notebook *llmops_demo.ipynb*, a bateria de testes executa as oito perguntas do conjunto de avaliação com as duas versões de prompt:

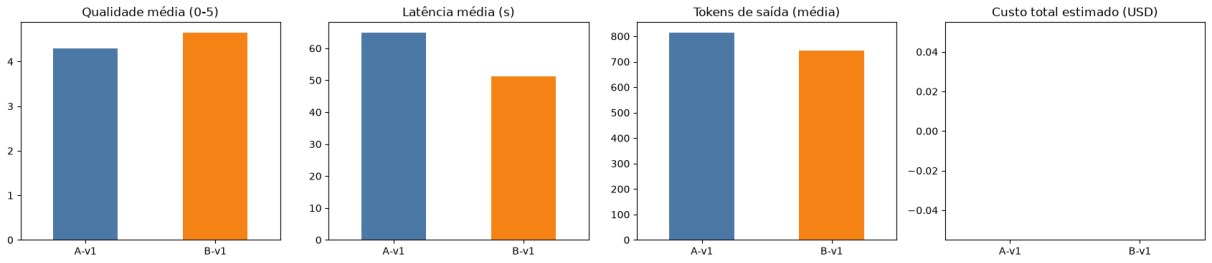
```
linhas, erros = core.rodar_bateria(versoes=("A", "B"), origem="notebook", progresso=mostrar_progresso)
print(f"\n{len(linhas)} execuções registradas, {len(erros)} erros.")
for e in erros:
    print("ERRO:", e)
```

Quando todas as execuções são concluídas com sucesso, são geradas 16 execuções, correspondentes às oito perguntas avaliadas com cada versão. Como as perguntas e o modelo são mantidos, a comparação se concentra nas diferenças entre os prompts.

Depois da avaliação, a função *resumo_por_prompt()* agrupa os registros por versão e calcula indicadores como a qualidade média, a latência média, a média de tokens de saída e o custo total estimado. Os resultados são apresentados em tabelas e gráficos no notebook, facilitando a identificação das diferenças entre as versões.

A função *recomendar()* utiliza os indicadores calculados para apontar qual versão apresentou a maior qualidade média, qual teve a menor latência e qual apresentou o menor custo estimado. A função também identifica situações em que esses critérios levam a escolhas diferentes. Com isso, a aplicação demonstra o ciclo de melhoria contínua do LLMOps: medir os resultados, comparar versões, analisar os pontos positivos e negativos e utilizar essas informações para orientar ajustes futuros.

Figura 2 – Comparação das métricas das versões A-v1 e B-v1 dos prompts



Fonte: *Elaborado pela autora com base nos resultados registrados no experimento (2026)*

Tabela 2 – Resultados da avaliação dos prompts

prompt_versao	execucoes	qualidade_media	latencia_media_s	tokens_entrada_medio	tokens_saida_medio	custo_total_usd
A-v1	16	4.2940	64.9374	19.50	814.5000	0.0
B-v1	12	4.6525	51.1511	46.25	744.5833	0.0

Fonte: *Elaborado pela autora com base nos resultados registrados no experimento (2026)*

Após a execução dos testes, foram obtidos resultados para as duas versões de prompt, apresentados na Figura 2 e na Tabela 2. A versão A-v1 contou com 16 execuções registradas, enquanto a B-v1 apresentou 12. A qualidade média foi de 4,2940 para A-v1 e 4,6525 para B-v1, indicando uma diferença de aproximadamente 0,36 ponto em uma escala de zero a cinco. Dessa forma, a versão B-v1 apresentou melhor resultado médio nesse critério.

Em relação à latência, a versão A-v1 apresentou média de 64,94 segundos, enquanto a B-v1 registrou 51,15 segundos, sendo aproximadamente 21,2% mais rápida. A média de tokens de saída também foi menor na B-v1, com 744,58 tokens, contra 814,50 da A-v1. Por outro lado, a média de tokens de entrada foi maior na B-v1, passando de 19,50 para 46,25 tokens, o que indica que suas instruções utilizaram mais tokens de entrada.

O custo total estimado foi igual a zero nas duas versões. Entretanto, como os preços estavam configurados como zero no sistema, esse resultado não permite concluir que as chamadas à API não tiveram custo real. Além disso, a diferença no número de execuções limita a comparação, pois as versões não foram avaliadas com a mesma quantidade de registros. Assim, embora a B-v1 tenha apresentado melhores médias de qualidade e latência, é recomendável completar os testes com as mesmas perguntas antes de tomar uma decisão definitiva sobre sua utilização.

Os resultados também mostraram que a alteração do prompt pode influenciar tanto a qualidade das respostas quanto o desempenho da aplicação. Neste experimento, a versão B-v1 apresentou maior qualidade média e menor latência, mas utilizou mais tokens de entrada. Como o número de execuções foi diferente entre as versões e a avaliação possui limitações, esses resultados devem ser interpretados como indicativos, e não como uma comprovação definitiva de superioridade. A realização de novos testes, com um conjunto equivalente de perguntas e configurações de custo adequadas, permitirá avaliar melhor os efeitos das alterações nos prompts.

4.3 RELATO DE EXPERIÊNCIA

O desenvolvimento da aplicação permitiu observar que a utilização de um modelo de linguagem envolve mais do que realizar uma requisição e apresentar a resposta ao usuário. Para demonstrar os conceitos de LLMOps, foi necessário organizar o fluxo de execução de modo que cada resposta pudesse ser identificada, avaliada e comparada com as demais.

A maior dificuldade encontrada durante o desenvolvimento do projeto foi a implementação da interface gráfica, principalmente devido à minha pouca experiência com

desenvolvimento front-end. Para superar essa dificuldade, utilizei a inteligência artificial como ferramenta de apoio durante essa etapa, orientando seu uso por meio de prompts elaborados por mim e acompanhando as sugestões geradas, de forma controlada e alinhada aos objetivos do projeto.

Outro desafio técnico desse tipo de aplicação foi na avaliação da qualidade das respostas. Diferentemente de problemas com resultados exatos, respostas geradas por modelos de linguagem variam muito em sua forma de apresentação e no nível de detalhamento. Por esse motivo, a aplicação combina a verificação automática de palavras-chave com a avaliação manual, permitindo considerar tanto critérios objetivos quanto a análise do conteúdo produzido. Ainda assim, a cobertura de palavras-chave não garante, por si só, que uma resposta esteja correta ou seja suficientemente clara.

Outro ponto importante é o acompanhamento dos custos e do desempenho. A latência pode variar conforme as condições da requisição e do serviço utilizado, enquanto a quantidade de tokens depende tanto do prompt quanto do tamanho da resposta. Além disso, quando a API não fornece os dados reais de utilização, as estimativas adotadas pelo sistema apresentam limitações. Por isso, é necessário distinguir os valores medidos daqueles que foram estimados ao interpretar os resultados.

A organização do código em funções também foi importante para separar a chamada ao modelo, o registro das execuções e a análise dos resultados. Essa divisão facilita a compreensão do fluxo e permite que a interface e o notebook utilizem a mesma lógica central, evitando a necessidade de implementar o processo de avaliação duas vezes.

5. PERSPECTIVAS FUTURAS E MERCADO

5.1 TENDÊNCIAS

A tendência é que o LLMOps se torne cada vez mais importante à medida que as empresas ampliem o uso de modelos de linguagem em suas aplicações. Nos próximos anos, espera-se o desenvolvimento de ferramentas mais especializadas para acompanhar o desempenho dos modelos, avaliar a qualidade das respostas e automatizar etapas de implantação e monitoramento, acompanhando a evolução das práticas de LLMOps (DEY, 2025). Também deve haver avanços na padronização dos métodos de avaliação, facilitando a comparação entre diferentes modelos e versões de uma aplicação.

Outro ponto relevante é a evolução das práticas de segurança e conformidade. Será necessário aprimorar os mecanismos de observabilidade para identificar alterações na qualidade das respostas, reduzir alucinações e evitar vazamentos de informações. Além disso,

o crescimento dos modelos multimodais, capazes de processar texto, imagens e áudio, deve ampliar o escopo do LLMOps e trazer novos desafios de gerenciamento. A busca por maior eficiência computacional também tende a incentivar o uso de modelos menores, otimização de recursos e estratégias para reduzir custos e latência.

5.2 ADOÇÃO NA INDÚSTRIA E DEMANDA POR PROFISSIONAIS

A adoção do LLMOps está relacionada à expansão do uso de inteligência artificial generativa em diferentes setores. No mercado financeiro, por exemplo, modelos de linguagem podem auxiliar na análise de documentos, no atendimento ao cliente e na consulta a informações. Na saúde, podem apoiar a organização de registros e a análise de literatura científica, enquanto no setor jurídico podem facilitar a pesquisa e a revisão de documentos. Empresas de tecnologia também utilizam esses modelos na criação de assistentes de programação, ferramentas de busca inteligente e aplicações baseadas em bases de conhecimento próprias.

Nesses cenários, o LLMOps contribui para que os modelos sejam utilizados de maneira mais confiável, com acompanhamento da qualidade das respostas, controle de custos, monitoramento de desempenho e aplicação de medidas de segurança. Quanto maior a dependência de uma aplicação em relação à inteligência artificial, maior a importância de manter processos que permitam identificar falhas e realizar melhorias.

Esse movimento também amplia a necessidade de profissionais que integrem conhecimentos de inteligência artificial, ciência de dados e engenharia de software. Além de desenvolver soluções com modelos de linguagem, esses profissionais precisam saber avaliar resultados, monitorar aplicações, gerenciar versões e otimizar recursos computacionais. Dessa forma, o LLMOps representa uma possibilidade de atuação para profissionais que desejam trabalhar na transição entre o desenvolvimento de soluções de IA e sua utilização em ambientes reais.

6. CONCLUSÃO E REFERÊNCIAS

O estudo do LLMOps permite compreender sua importância para o desenvolvimento e a manutenção de aplicações baseadas em grandes modelos de linguagem. A tecnologia amplia as práticas já utilizadas em DevOps e MLOps, incorporando mecanismos específicos para acompanhar a qualidade das respostas, controlar custos, monitorar o desempenho e gerenciar versões de prompts e modelos.

A partir dessa análise, considera-se que o LLMOps é uma abordagem viável e útil, principalmente para aplicações que dependem continuamente de modelos de linguagem e

precisam oferecer respostas confiáveis em escala. Seus principais benefícios incluem o aumento da eficiência, a melhoria da escalabilidade e a redução de riscos relacionados à segurança e à privacidade (RED HAT, 2024). Entretanto, sua implementação exige planejamento, recursos e critérios adequados de avaliação, uma vez que essas práticas não eliminam completamente problemas como alucinações e respostas inconsistentes.

Portanto, o LLMOps representa um recurso importante para aproximar o desenvolvimento de soluções de inteligência artificial das necessidades de operação em ambientes reais. Sua adoção permite acompanhar o comportamento das aplicações e orientar melhorias com base nos resultados observados. Dessa forma, sua utilidade está não apenas em facilitar a implantação de modelos de linguagem, mas também em contribuir para que seu uso seja mais eficiente, controlado e responsável.

CHOUDHARY, Abhishek. **O que é LLMOps?** Um Guia para Operações de Grandes Modelos de Linguagem. TrueFoundry, 21 maio. 2026. Disponível em: https://www.truefoundry.com/pt/blog/what-is-llmops. Acesso em: 19 out. 2026.

CHOUDHARY, Abhishek. **LLMOps vs MLOps:** Um Guia de Comparação Completo. TrueFoundry, 21 maio 2026. Disponível em: https://www.truefoundry.com/pt/blog/llmops-vs-mlops. Acesso em: 15 out. 2026.

COURSERA STAFF. **MLOps Engineer:** Roles, Skills, and Career Path. Coursera, 25 Aug. 2026. Disponível em: https://www.coursera.org/articles/mlops-engineer. Acesso em: 20 out. 2026.

DATACAMP. **LLMOps essentials:** um guia prático para operacionalizar modelos de linguagem grandes. DataCamp Blog, 17 set. 2026. Disponível em: https://www.datacamp.com/pt/blog/llmops-essentials-guide-to-operationalizing-large-language-models. Acesso em: 10 out. 2026.

DEEPLEARNING.AI. **LLMOps:** Introduction. DeepLearning.AI, [2026]. Disponível em: https://learn.deeplearning.ai/courses/llmops/lesson/jupuw/introduction. Acesso em: 20 out. 2026.

EQUIPE DA DATABRICKS. **MLOps vs DevOps:** Um Guia Prático para Cientistas de Dados e Equipes de TI. Blog Databricks, [2026]. Disponível em: https://www.databricks.com/br/blog/mlops-vs-devops. Acesso em: 15 out. 2026.

HOLSEDL, Damjan (maint.). **Articles & Guides:** Running LLMs in Production. LLMOps.si, 2026. Disponível em: https://llmops.si/articles/. Acesso em: 20 out. 2026.

DEY, Soumava. The evolution of LLMOps: from DevOps to MLOps and beyond. *Medium*, 6 mar. 2025. Disponível em: https://medium.com/@soumavadey/the-evolution-of-llmops-from-devops-to-mlops-and-beyond-b8a19536d3cd. Acesso em: 21 out. 2026.

MICROSOFT. *AI Playbook: LLMOps workflow*. [S. l.], [s. d.]. Disponível em: https://learn.microsoft.com/en-us/ai/playbook/assets/images/llmops-workflow.png. Acesso em: 21 out. 2026.

RED HAT. **O que é LLMOps?** Red Hat, 8 fev. 2024. Disponível em: https://www.redhat.com/pt-br/topics/ai/llmops. Acesso em: 10 out. 2026.

GLOSSÁRIO

- **LLM (*Large Language Model*):** modelo de inteligência artificial treinado para compreender e gerar linguagem natural.
- **DevOps:** abordagem que integra desenvolvimento e operações de software por meio de automação, colaboração e monitoramento.
- **MLOps (*Machine Learning Operations*):** práticas voltadas ao gerenciamento do ciclo de vida de modelos de aprendizado de máquina.
- **Prompt:** instrução ou conjunto de informações fornecidas ao modelo para orientar sua resposta.
- **Engenharia de prompts:** processo de elaboração e avaliação de instruções para melhorar os resultados gerados pelo modelo.
- **RAG (*Retrieval-Augmented Generation*):** técnica que combina a recuperação de informações externas com a geração de respostas por um LLM.
- **Inferência:** etapa em que o modelo utiliza o conhecimento adquirido durante o treinamento para gerar uma resposta a partir de uma entrada.

- **Latência:** tempo necessário para que o sistema processe uma solicitação e produza uma resposta.
- **Observabilidade:** capacidade de acompanhar o funcionamento do sistema por meio de métricas, registros e rastreamento das execuções.
- **Alucinação:** geração de informações incorretas ou não fundamentadas pelo modelo.
- **Fine-tuning:** ajuste de um modelo previamente treinado utilizando dados adicionais para adaptá-lo a uma tarefa específica.